Classificação de Textos com o Algoritmo Perceptron

<u>Aula 05</u>: Introdução ao Processamento de Linguagem Natural (PLN)

Prof. Eduardo Corrêa Gonçalves

16/03/2026

Sumário

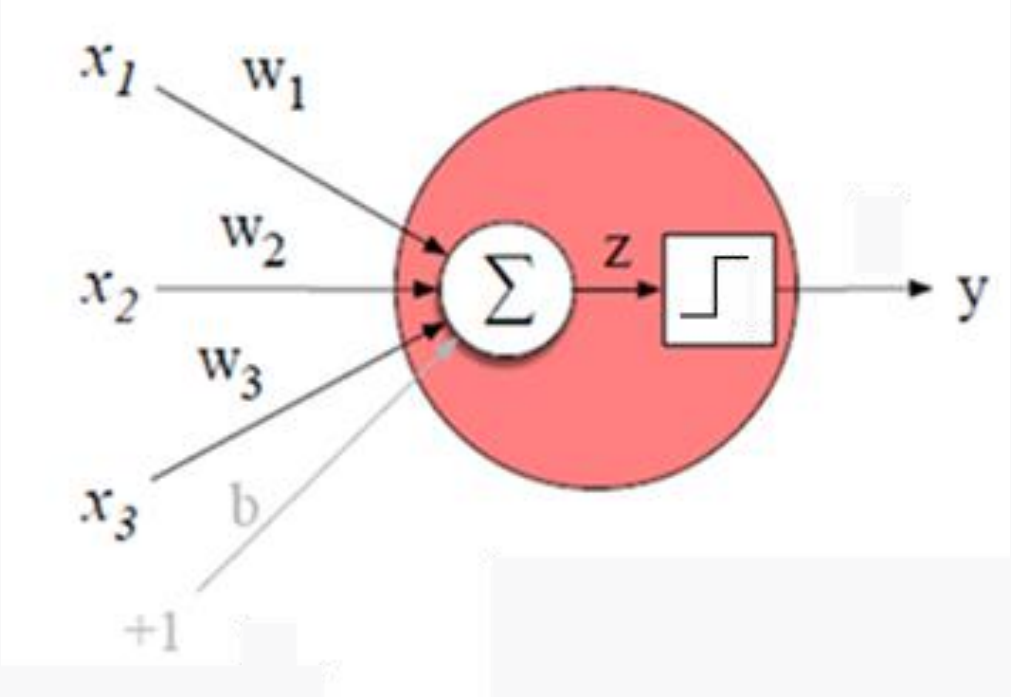
Introdução

Perceptron

- Estrutura de um Perceptron
- Perceptron – etapa de classificação
- Perceptron - treinamento

Introdução (1/4)

- **O que é Perceptron ?**
  - É um **método linear** que pode ser utilizado para treinar um **classificador binário**.
    - Abaixo um esquema de um classificador Perceptron para um problema de classificação com 3 *features* x_1, x_2 e x_3 (logo detalharemos essa estrutura...)



Introdução (2/4)

- **Perceptron – método linear** que pode ser utilizado para treinar um **classificador binário**.

  - **Classificador binário** é aquele usado em problemas envolvendo **2 classes**, tais como:
    - **Analisador de Sentimentos sem "neutro"**
      - classes: "positivo" x "negativo"
    - **Classificação de Spam**
      - classes: "spam" (e-mail é spam) x "normal" (e-mail é normal)
    - **Detecção de Discurso de Ódio**
      - classes: "sim" (texto é ofensivo) x "não" (não é)
    - **Detecção de Posicionamentos (*stance detection*)**
      - classes: "sim" (posição favorável) x "não" (desfavorável)

Introdução (3/4)

- **Perceptron – método linear** que pode ser utilizado para treinar um **classificador binário**.
  - **Método linear** é aquele que gera um **hiperplano** como solução.
    - Ou seja, a fronteira identificada pelo Perceptron para separar as duas classes será plana, sem curvatura...
      - (alguma "coisa" linear é algo que está relacionado a uma reta)
    - **Exemplo**:
      - Em um problema que envolva 3 *features* x_1, x_2 e x_3, o Perceptron poderia gerar como solução uma equação como:

$$5x_1 + 3x_2 - 4x_3 + 1 = 0$$

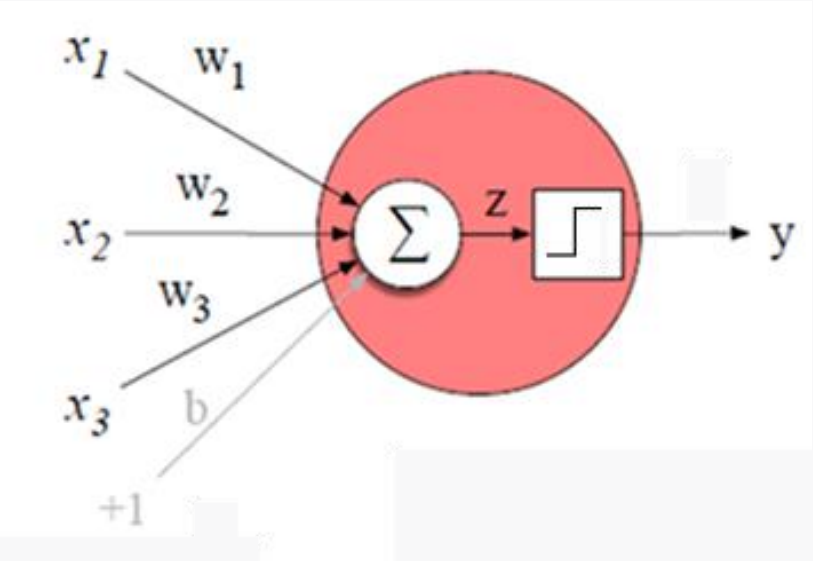
      - Essa equação representa o modelo de classificação !!!

Introdução (4/4)

- **Como todo algoritmo de classificação, o Perceptron tem 2 fases:**
  - **Treinamento**
    - Fase em que o modelo é construído a partir da base de treino.
    - No caso do Perceptron, o mais comum é utilizar o método do gradiente descendente e a minimização dos erros quadráticos para o treinamento.
  - **Classificação**
    - Fase em que o modelo construído na etapa de treinamento é utilizado para classificar um novo documento *d*.
  - *Vamos começar examinando a estrutura do Perceptron, depois como funciona a fase de classificação e, por fim, o treinamento*.

Estrutura do Perceptron (1/6)

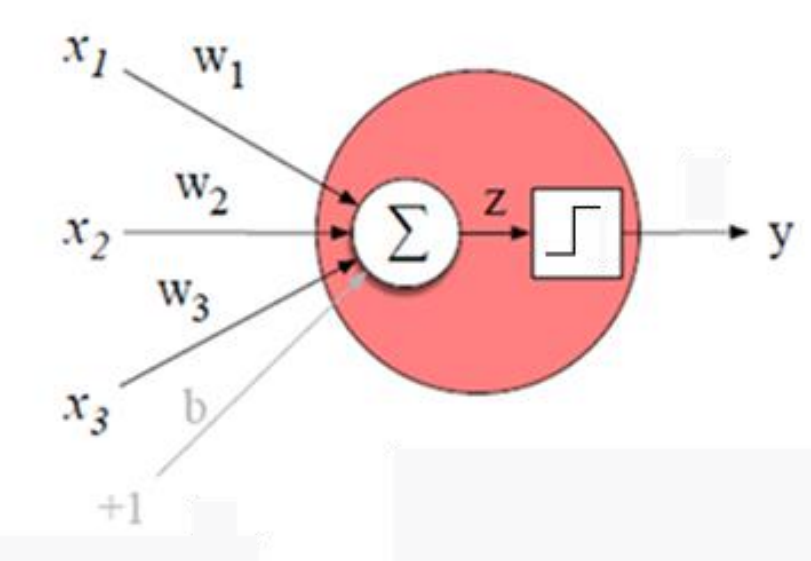
- **O Perceptron é uma estrutura bem "basicona" que:**
  - Recebe um conjunto de valores reais x_1, x_2, ... x_n como entrada.
    - Em nossa matéria, esses valores representarão as ***features**** de um documento *d*.
  - Executa uma computação sobre esses valores em seu núcleo (*área colorida da imagem*).
  - Produz uma saída *y*. A saída corresponderá a **classe predita** para o documento *d*.



** Em nosso curso de PLN o Perceptron será usado para classificar textos (obviamente!). Porém, é possível criar qualquer tipo de classificador com o algoritmo (classificador de imagens, classificador de crédito de clientes, classificador de fraudes bancárias etc.).*

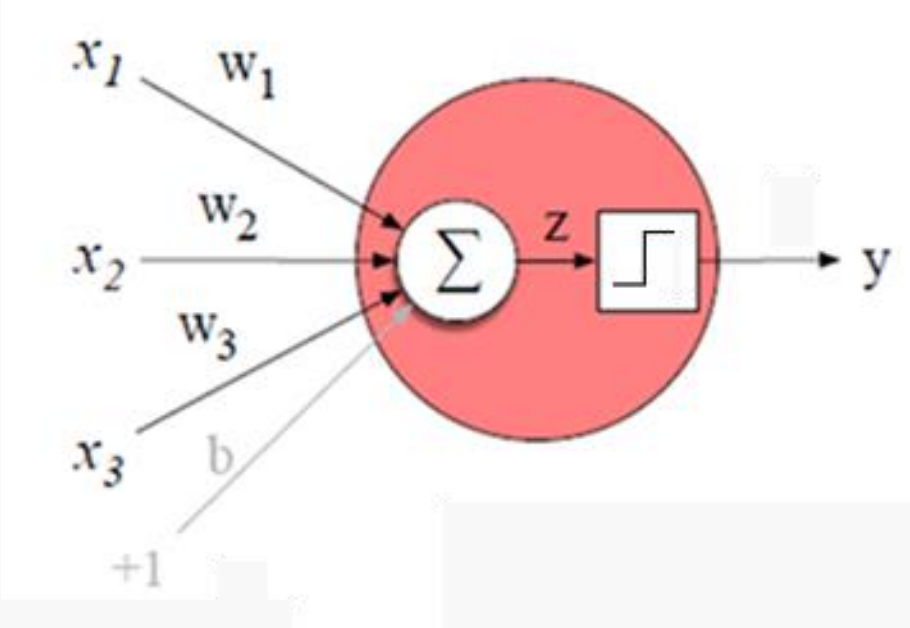
Estrutura do Perceptron (2/6)

- **O núcleo do Perceptron**
  - Em seu núcleo, o Perceptron realiza:
    - A soma ponderada das entradas (soma cada $w_i x_i$, i.e., cada entrada x_i multiplicada por seu peso w_i) mais o termo de viés *b* (*bias term*).
    - O resultado z é chamado de entrada (*input*) da unidade.
    - Em seguida, ele aplica uma função degrau sobre *z* para gerar a saída *y*.



Estrutura do Perceptron (3/6)

- **<u>Exemplo</u>: Cálculo da entrada *z* e da saída *y***
  - Dado o Perceptron abaixo, suponha que:
    - $x_1 = 3$; $x_2 = 2$; $x_3 = 0$
      - Esses são os **valores das *features*** extraídas de um documento, que podemos representar como o vetor **x** = [3, 2, 0]
    - $w_1 = 2.5$; $w_2 = -5.0$; $w_3 = 2.0$
      - Esses são os **pesos do modelo**, que podemos representar como o vetor **w** = [2.5, -5.0, 2.0]
    - $b = 0.1$ (termo de viés)
  - Como calcular o valor de *z* (entrada do Perceptron) ?
  - Como calcular o valor de *y* (saída do Perceptron) ?



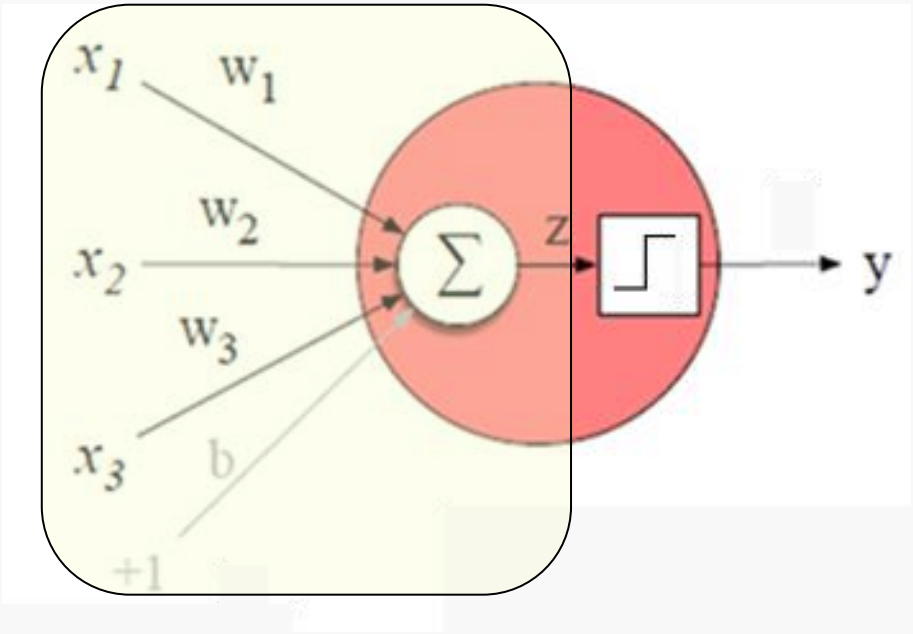
Estrutura do Perceptron (4/6)

- **<u>Exemplo (parte 1):</u> cálculo da entrada *z***
  - $x_1 = 3$; $x_2 = 2$; $x_3 = 0$
  - $w_1 = 2.5$; $w_2 = -5.0$; $w_3 = 2.0$
  - $b = 0.1$

  x = [3, 2, 0]
  w = [2.5, -5.0, 2.0]

  - $z = 2.5 \times 3 + -5.0 \times 2 + 2.0 \times 0 + 0.1 =$ **<u>-2.4</u>**

$$z = \left(\sum_{i=1}^{n} w_i x_i\right) + b$$



Estrutura do Perceptron (5/6)

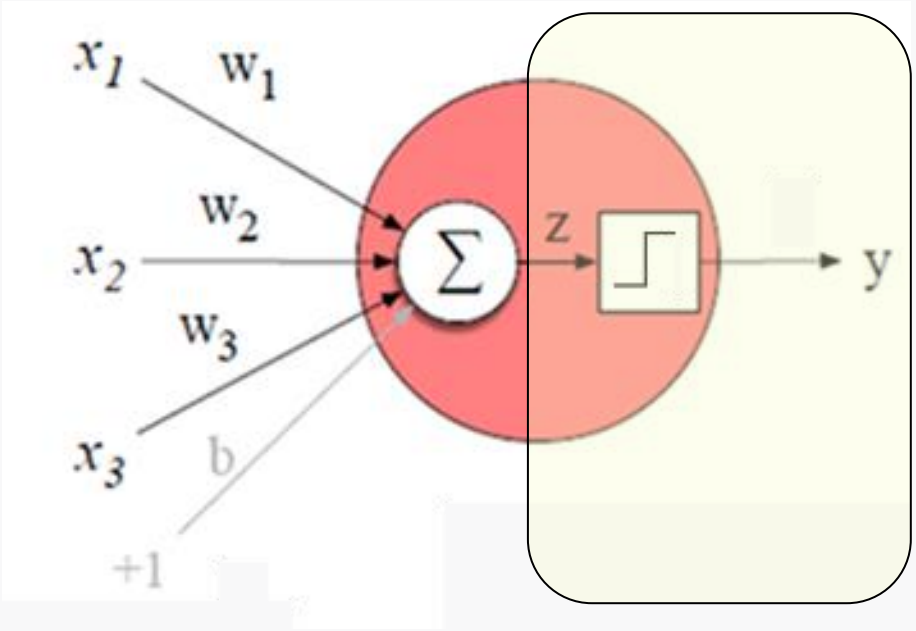
- **<u>Exemplo (parte 2):</u> cálculo da saída *y***
  - O Perceptron é extremamente simples.
    - Ele aplica a **função degrau** sobre *z* (a entrada) p/ gerar *y* (a saída)
    - A função degrau possui saída binária:
    - $\sqcap(z) \begin{cases} 0, \text{ se } \mathbf{wx} + b \leq 0 \\ 1, \text{ se } \mathbf{wx} + b > 0 \end{cases}$

- **Obs.**: o Perceptron é <u>mais simples</u> do que as atuais unidades computacionais que formam as redes neurais (RNs) modernas.
  - As unidades das RNs modernas assemelham-se a regressores logísticos.
  - Uma RN profunda pode ser vista como uma estrutura com dezenas ou milhares de regressores logísticos distribuídos em camadas e interconectados !!!
  - Por isso a disciplina de MLG é tão importante para quem quer trabalhar com RNs (assim como cálculo, álgebra linear, estatística multivariada...)

Estrutura do Perceptron (6/6)

- **<u>Exemplo (parte 2):</u> cálculo da saída *y***
  - $x_1 = 3$; $x_2 = 2$; $x_3 = 0$ — **x** = [3, 2, 0]
  - $w_1 = 2.5$; $w_2 = -5.0$; $w_3 = 2.0$ — **w** = [2.5, -5.0, 2.0]
  - $b = 0.1$
  - $z = 2.5 \times 3 + -5.0 \times 2 + 2.0 \times 0 + 0.1 = \underline{\mathbf{-2.4}}$
  - $y = \sqcap(-2.4) = \underline{\mathbf{0}}$
    - Sendo assim, o modelo classificou o documento com a classe 0.

$$\sqcap(z) \begin{cases} 0, \text{ se } \mathbf{wx} + b \leq 0 \\ 1, \text{ se } \mathbf{wx} + b > 0 \end{cases}$$



Classificação com o Perceptron (1/5)

- **Objetivo:** Empregar um classificador Perceptron já treinado para realizar decisão binária sobre a classe de um documento.
  - Dado um novo documento com *n features*: **x** = $[x_1, x_2, ..., x_n]$.
  - A saída y do classificador poderá ser:
    - 1 (doc. é membro da classe); ou
    - 0 (não é membro).

- **Exemplo**: Classificação de Sentimentos (**Positivo** x **Negativo**)
  - **x** = *features* extraídas de um documento.

    A saída y do classificador poderá ser:
    - y=1 se o Perceptron classificar o doc. como sent. positivo.
    - y=0 se o Perceptron classificar o doc. como sent. negativo.

    - Recorde que a saída sempre será 1 ou 0 já que é gerada pela função degrau.

Classificação com o Perceptron (2/5)

- **<u>Exemplo</u> – Classificador de Sentimentos**
  - **Cenário**:
    - Considere que ***2 features*** [x_1, x_2] são extraídas do documento.
      - x_1 = total de palavras positivas no texto
      - x_2 = total de palavras negativas
    - Suponha que o classificador Perceptron **já foi treinado** (*ainda não foi mostrado como fazer isso!*), e **w** e *b* sejam os seguintes:
      - **w** = [2.5, -5.0]
      - *b* = 0.1
    - Vamos mostrar como classificar o documento, já pré-processado, representado pelo vetor **x** = [1, 3]
      - 1 palavra positiva e 3 palavras negativas
    - A saída *y* será 1 (sent. positivo) ou 0 (sent. Negativo)

* *Veja que nesse exemplo, os documentos não são convertidos para Bag of Words (embora o Perceptron também aceite o uso dessa estrutura)*

Classificação com o Perceptron (3/5)

- **Mas antes ... <u>Observe os pesos</u> !!!**

- **w** = [2.5, -5.0]

$w_1 = 2.5$

Indica o quão importante é a *feature* x_1 (total de palavras positivas, como "great", "nice" etc.) para classificar o texto como positivo.

Veja que $w_1 > 0$

- x_1 = total de palavras positivas no texto
- x_2 = total de palavras negativas

IBGE
Escola Nacional de Ciências Estatísticas ENCE

Classificação com o Perceptron (4/5)

- **Mas antes ... <u>Observe os pesos</u> !!!**

- **w** = [2.5, -5.0]

- x_1 = total de palavras positivas no texto
- x_2 = total de palavras negativas

w_2 = -5.0

Indica que palavras negativas (x_2) estão associadas negativamente com a decisão do texto ter sentimento positivo...

... e que isso parece ser duas vezes mais importante do que a relação entre palavras positivas e textos positivos.

Veja que $w_2 < 0$

IBGE | Escola Nacional de Ciências Estatísticas ENCE

Classificação com o Perceptron (5/5)

- **Exemplo – Classificando um Novo Documento**
  - Novo documento: **x** = [1, 3]
  - Parâmetros do Modelo: **w** = [2.5, -5.0] e *b* = 0.1

  - **PARTE 1**: cálculo da entrada *z*
    - Dado que **w** e **x** são vetores, podemos representar o cálculo de *z* com a notação do produto escalar da Álgebra Linear: $z = \mathbf{w} \,.\, \mathbf{x} + b$
    - *z* = [2.5, -5.0] . [1, 2] + 0.1 = **-12.4**

  - **PARTE 2**: cálculo da saída *y*
    - *y* = $\sqcap$ (-12.4) = **0**

  - Dessa forma, como a saída y = 0, então **x** é classificado como **negativo**.

Treinamento do Perceptron – visão geral (1/3)

- **Como os <u>parâmetros</u> w (vetor de pesos) e b (viés) são aprendidos no Perceptron?**
  - O Perceptron é um classificador binário, logo precisa ser treinado a partir de uma **base rotulada**.
    - BD de treinamento em que cada objeto x está rotulado com sua a classe y (0 ou 1)
  - Quando um Perceptron é usado para classificar um objeto, ele computa $\hat{y}$ (classe estimada do verdadeiro y).
  - Desta forma, o que deseja-se é:
    - Aprender os parâmetros **w** e *b* capazes de fazer com que o $\hat{y}$ de cada objeto de treino seja o mais próximo possível de seu y verdadeiro.

Treinamento do Perceptron – visão geral (2/3)

- **Para tal, o Perceptron usa <u>2 componentes</u> no treinamento**

1. Uma **função de custo** a ser minimizada
   - Ela avalia o quão distante $\hat{y}$ está do *gold label* y de maneira geral, ou seja, considerando todos os objetos de treino.
   - Adotaremos a minimização da **Soma de Quadrados do Erro** (SQE)

2. Um **algoritmo** para otimizar a função de perda
   - Realiza, de forma iterativa, a atualização dos pesos com o intuito de minimizar a função de custo.
   - Usaremos o **Gradiente Descendente Estocástico** (GD)

- Existem diversas funções de custo e algoritmos de otimização. A SQE e GD são adotadas em muitos livros, como de Oliveira e Melo Junior (2024).

Treinamento do Perceptron – visão geral (3/x)

- **Notação**
  - Os slides a seguir abordarão o treinamento do Perceptron com a SQE e GD. A seguinte notação é adotada:
    - D → base de dados de treinamento de dimensão $m \times n$
      - m → número de documentos
      - n → número de *features*
      - Cada documento representado na base é da forma $(\mathbf{x}, y)$, onde $\mathbf{x}$ é o vetor de *features* e y o rótulo de classe.
    - $(x^{(k)}, y^{(k)})$ → *k*-ésimo documento da base de treinamento
      - $x^{(k)}$ → *features* do k-ésimo documento
      - $y^{(k)}$ → classe do k-ésimo documento
    - x_i → *i*-ésima feature de $\mathbf{x}$

Soma do Quadrado dos Erros – SQE (1/6)

- **<u>Componente 1</u> do treinamento: Função de Custo**
  - Dado um objeto de treino *k*, o **erro** $e^{(k)}$ na saída do Perceptron para esse objeto é obtido pela diferença entre o valor real $y^{(k)}$ da classe do objeto e o valor predito $\hat{y}^{(k)}$.
    - $e^{(k)} = y^{(k)} - \hat{y}^{(k)}$
  - A **SQE** é uma maneira de quantificar o erro considerando todos os objetos da BD.
    - Dada uma base de treino com *m* documentos, a SSE é obtida elevando ao quadrado as diferenças entre o valor real e o valor predito (erro) de todos os documentos e, em seguida, somando esses quadrados.

$$SQE = \sum_{k=1}^{m} \left(y^{(k)} - \hat{y}^{(k)}\right)^2$$

como $y^{(k)} - \hat{y}^{(k)} = e^{(k)}$, podemos também representar por: →

$$SQE = \sum_{k=1}^{m} \left(e^{(k)}\right)^2$$

Soma do Quadrado dos Erros – SQE (2/6)

- **Quanto menor o valor da SQE, melhor**
  - Um valor baixo, indica que o Perceptron está gerando estimativas próximas das classes reais.
    - Se o Perceptron não errar nada, a SQE obtida será 0.
    - Quanto mais distantes os valores preditos forem dos reais, maior será a SQE.
    - Vamos ver alguns exemplos utilizando a BD abaixo
      - (*É um exemplo clássico que corresponde à função AND*)

$$SQE = \sum_{k=1}^{m} \left(y^{(k)} - \hat{y}^{(k)}\right)^2$$

AND		
x1	x2	y
0	0	0
0	1	0
1	0	0
1	1	1

Soma do Quadrado dos Erros – SQE (3/6)

- **SQE – exemplos**
  - Observe que as classes reais (coluna y) são $y^{(1)}=0$, $y^{(2)}=0$, $y^{(3)}=0$ e $y^{(4)}=1$

AND		
x1	x2	y
0	0	0
0	1	0
1	0	0
1	1	1

- **<u>Exemplo 1</u> – o Perceptron <u>acertou</u> tudo!** ☺
  - Suponha que o perceptron previu $\hat{y}^{(1)}=0$, $\hat{y}^{(2)}=0$, $\hat{y}^{(3)}=0$ e $\hat{y}^{(4)}=1$
    - Ou seja, ele acertou tudo!
    - A SQE nesse caso resulta em <u>zero</u>:

$$SQE = \sum_{k=1}^{m} \left(y^{(k)} - \hat{y}^{(k)}\right)^2 = (y^{(1)} - \hat{y}^{(1)})^2 + (y^{(2)} - \hat{y}^{(2)})^2 + (y^{(3)} - \hat{y}^{(3)})^2 + (y^{(4)} - \hat{y}^{(4)})^2 =$$
$$= (0 - 0)^2 + (0 - 0)^2 + (0 - 0)^2 + (1 - 1)^2 = \mathbf{0.0}$$

Soma do Quadrado dos Erros – SQE (4/6)

- **SQE – exemplos**
  - Classes reais (coluna y) → $y^{(1)}=0$, $y^{(2)}=0$, $y^{(3)}=0$ e $y^{(4)}=1$

AND		
x1	x2	y
0	0	0
0	1	0
1	0	0
1	1	1

- **<u>Exemplo 2</u> – o Perceptron <u>errou</u> tudo!** ☹
  - Suponha agora que o perceptron previu $\hat{y}^{(1)}=1$, $\hat{y}^{(2)}=1$, $\hat{y}^{(3)}=1$ e $\hat{y}^{(4)}=0$
    - Ou seja, ele errou tudo!
    - Nesse caso, a SQE terá **valor máximo** <u>para a base de dados</u>

$$SQE = \sum_{k=1}^{m} \left(y^{(k)} - \hat{y}^{(k)}\right)^2 = (y^{(1)} - \hat{y}^{(1)})^2 + (y^{(2)} - \hat{y}^{(2)})^2 + (y^{(3)} - \hat{y}^{(3)})^2 + (y^{(4)} - \hat{y}^{(4)})^2 =$$
$$= (0 - 1)^2 + (0 - 1)^2 + (0 - 1)^2 + (1 - 0)^2 = \mathbf{4.0}$$

Soma do Quadrado dos Erros – SQE (5/6)

- **SQE – exemplos**
  - Classes reais (coluna y) → $y^{(1)}=0$, $y^{(2)}=0$, $y^{(3)}=0$ e $y^{(4)}=1$

AND		
x1	x2	y
0	0	0
0	1	0
1	0	0
1	1	1

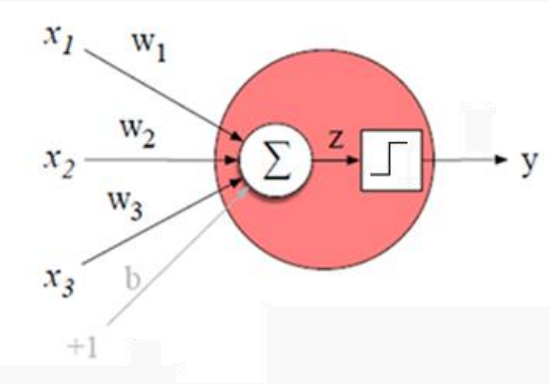
- **<u>Exemplo 3</u> – o Perceptron acertou metade!** 😐
  - Dessa vez, considere que o perceptron previu $\hat{y}^{(1)}=1$, $\hat{y}^{(2)}=1$, $\hat{y}^{(3)}=0$ e $\hat{y}^{(4)}=1$
    - Errou as 2 primeiras predições e acertou as 2 últimas.
    - A SQE será computada por:

$$SQE = \sum_{k=1}^{m} \left(y^{(k)} - \hat{y}^{(k)}\right)^2 = (y^{(1)} - \hat{y}^{(1)})^2 + (y^{(2)} - \hat{y}^{(2)})^2 + (y^{(3)} - \hat{y}^{(3)})^2 + (y^{(4)} - \hat{y}^{(4)})^2 =$$
$$= (0 - 1)^2 + (0 - 1)^2 + (0 - 0)^2 + (1 - 1)^2 = \mathbf{2.0}$$

- Como toda função de custo, a SQE tem suas vantagens e desvantagens, conforme discutido em https://estatisticafacil.org/glossario/o-que-e-sum-of-squared-errors-soma-dos-erros-quadraticos/

Soma do Quadrado dos Erros – SQE (6/6)

- **O erro é função dos pesos**
  - Se os pesos do Perceptron estiverem com "bons valores"...
    - ... o valor calculado para *z* ficará "bom"
    - ... e o erro associado a cada objeto *k* será baixo
    - ... e, por consequência, a SQE ficará baixa.



$$z = \left(\sum_{i=1}^{n} w_i x_i\right) + b$$

  - Mas se os pesos forem "ruins", o erro vai aumentar (o Perceptron vai errar mais classificações)...
    - ... com o consequente aumento da SQE.

$$SQE = \sum_{k=1}^{m} \left(y^{(k)} - \hat{y}^{(k)}\right)^2$$

  - Precisamos então de um processo para alcançar o valor desejado para os pesos.
    - É aí que entrará o Gradiente Descendente

Gradiente Descendente – Intuição (1/5)

- **O que é gradiente descendente (GD) ?**
  - É um processo iterativo para alcançar o valor desejado para os pesos baseado no gradiente do erro.
    - **Melhor dizendo**: o objetivo do GD é determinar os **parâmetros** que minimizam o erro do modelo (Perceptron, RNFF ou outra RN) considerando todos os objetos de treino.
      - Os parâmetros são denotados por θ (convenção nas áreas de IA e PLN).
      - No caso do Perceptron, $\theta = \mathbf{w}, b$

Gradiente Descendente – Intuição (2/5)

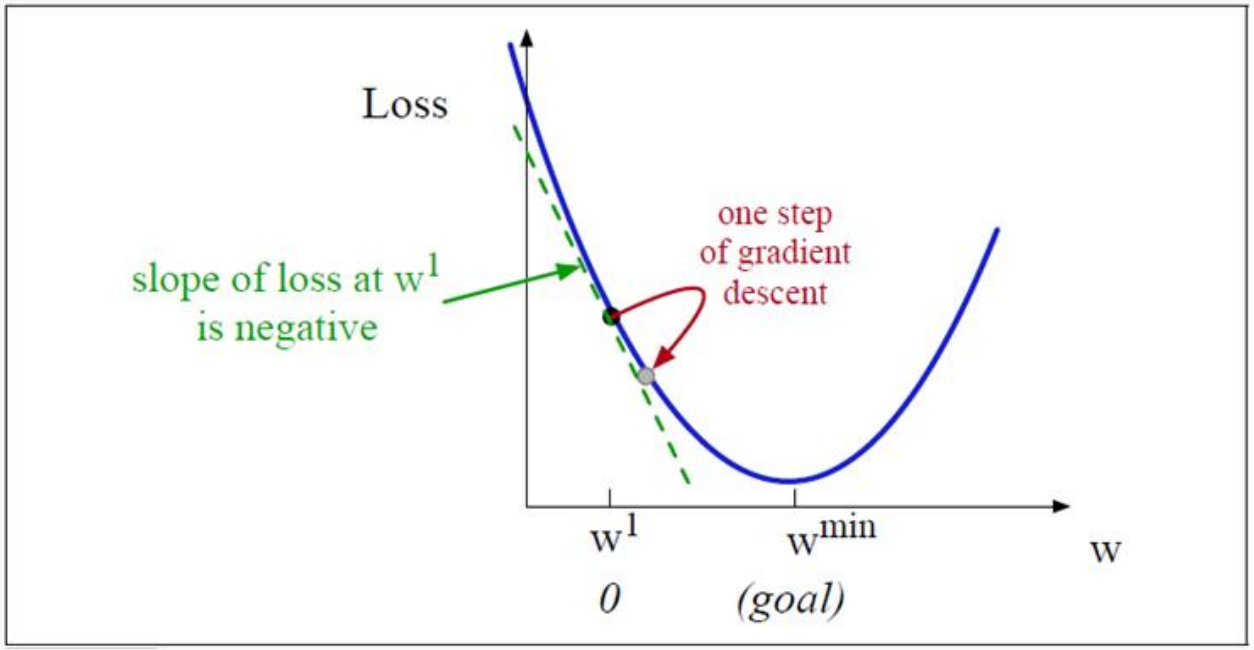
- **Como o GD funciona ?**
  - Em um dado ponto de referência (ponto corrente), o GD descobre em que direção, no espaço de parâmetros θ, a inclinação da função de custo (denotada por L) cresce de forma mais acentuada.
  - Depois de descobrir, ele se move, a partir do ponto corrente, na direção oposta.
    - Pois o objetivo é chegar ao mínimo de custo e não ao máximo !
  - Nos slides a seguir, representaremos o valor da função de custo para um documento específico $\mathbf{x}^{(k)}$ como $L\left(f\left(\boldsymbol{x}^{(k)};\theta\right), y^{(k)}\right)$
    - Acima, $\hat{y}^{(k)}$ é representado como $f(\mathbf{x}^{(k)},\theta)$
    - A notação explícita o fato de que a função de perda L é **parametrizada** por θ.

Gradiente Descendente – Intuição (3/5)

- **GD – Intuição**
  - Imagine que você é um alpinista que quer descer uma montanha o mais rápido possível.
  - Você olha 360º em volta e identifica onde o terreno é mais inclinado
  - E desce alguns passos nessa direção (*no mundo das RNs, o passo é chamado de taxa de aprendizagem e denotado por* η)
  - E repete o processo, até conseguir completar a descida.

Gradiente Descendente – Intuição (4/5)

- **GD – Exemplo**
  - O exemplo abaixo mostra um passo do GD em um problema de um único parâmetro *w*.
    - **Dados**: um valor inicial aleatório w^1 para *w;* e a função de custo L com a forma abaixo.
    - O GD nos diz **como** devemos **nos mover** para **alcançar o mínimo**:
      - para a **esquerda** (fazendo w^2 ser menor do que w^1) ?
      - **ou** para a **direita** (fazendo w^2 ser maior do que w^1) ?

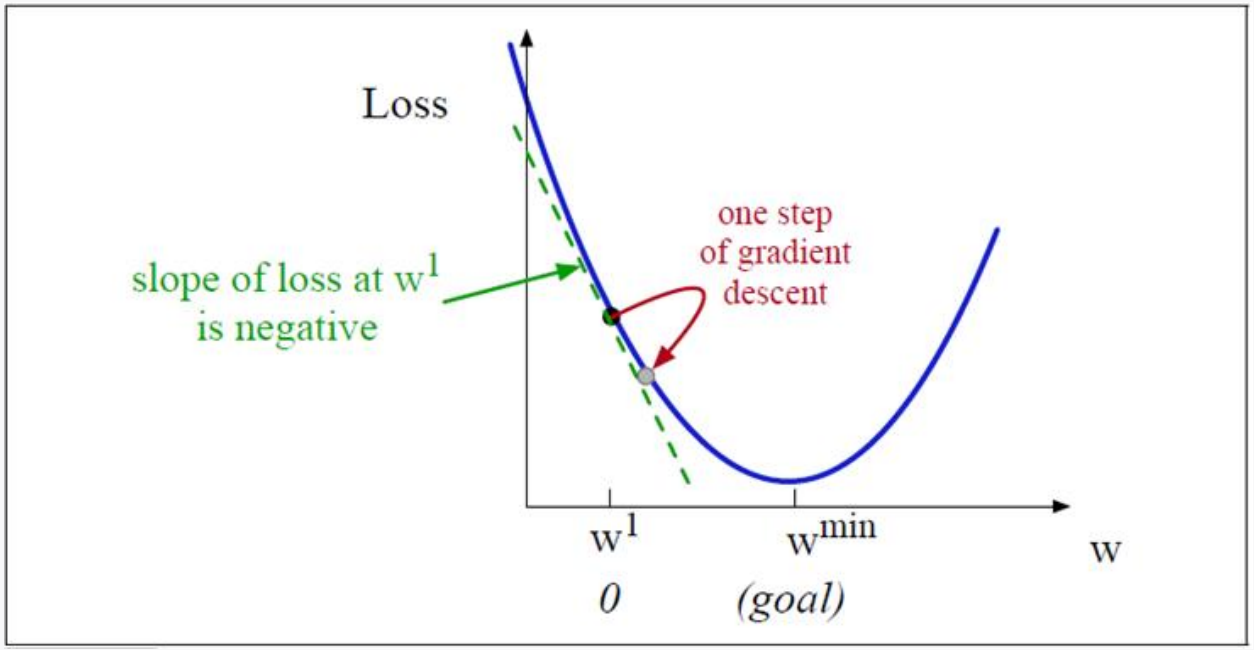


Fonte:
Jurafsky e Martin (2026)

IBGE
Escola Nacional de Ciências Estatísticas ENCE

Gradiente Descendente – Intuição (5/5)

- **GD – Exemplo**
  - A reta pontilhada indica a taxa de variação de L (*Loss*, eixo Y) em relação a w (eixo X) no instante w^1.
  - Como a inclinação da reta no ponto w^1 é negativa nos **moveríamos** para a **direita** (w^2 deve ser maior do que w^1).



IBGE
Escola Nacional de Ciências Estatísticas ENCE

Gradiente Descendente – Intuição (1/8)

- **Mas <u>como e o quanto</u> devemos nos mover ?**
  - O GD responde a essa questão encontrando o **gradiente** da função de perda no ponto corrente e fazendo o movimento na direção contrária.
  - Gradiente de uma função multivariada (com muitas variáveis) → vetor apontando na direção da inclinação mais acentuada da função.
  - A quantidade a ser movida é controlada pela **taxa de aprendizagem** η.
    - η é um **hiperparâmetro** que é especificado pelo usuário.
    - Um valor alto significa que o movimento de *w* será mais largo a cada passo (vai "andar mais rápido", porém com risco de passar do mínimo)

Gradiente Descendente – Intuição (2/8)

- **Mas <u>o quanto</u> devemos nos mover ?**
  - Para uma função com muitas variáveis (multivariada), a mudança no valor dos parâmetros θ, é determinada por:
  - $\boldsymbol{\theta^{t+1} = \theta^{t} - \eta \nabla L(f(x;\theta), y)}$
    - θ^{t+1} → próximo valor de θ (valor no instante t+1)
    - θ^{t} → valor atual de θ
    - η → taxa de aprendizagem (controla o tamanho do "passo" que será dado, normalmente valor no intervalo $0 < \eta \leq 1$)
    - $\nabla L(f(x;\theta), y)$ → gradiente da função de custo no ponto corrente.
  - **Ou seja**: a partir de onde estou agora (θ^{t}), devo fazer η movimentos na direção oposta a que o gradiente indica ($-\nabla L(f(\boldsymbol{x};\theta), y)$) para chegar ao novo local (θ^{t+1})

Gradiente Descendente – Intuição (3/8)

- **Gradiente de uma Função**
  - No exemplo dos slides 30 e 31, tínhamos somente uma variável.
    - Foi então possível pensar informalmente no gradiente como a inclinação de uma reta.
    - Dessa forma, para achar o mínimo, podemos nos mover para esquerda ou direita.

  - Porém, em problemas reais, o número de variáveis é muito maior !!!
    - Por isso, na verdade, precisamos saber onde devemos nos mover, no espaço *N*-dimensional (dos *N* parâmetros que compõem θ).

    - Sendo assim, o gradiente será um **vetor** ...
      - ... que expressa os componentes direcionais da inclinação mais acentuada entre cada uma das *N* dimensões.

Gradiente Descendente – Intuição (4/8)

- **Gradiente de uma Função**
  - Para cada dimensão w_i em **w** (mais o intercepto *b*), o vetor gradiente terá um componente que nos diz a inclinação com respeito a essa dimensão.
  - Em cada dimensão w_i, expressamos a **inclinação** como a **derivada parcial** $\frac{\partial}{\partial w_i}$ **da função de custo**.
    - "*O quanto uma pequena mudança em w_i influencia na função de custo L como um todo?*"

Gradiente Descendente – Intuição (5/8)

- **Gradiente de uma Função**
  - Usa-se o símbolo ∇ para referir ao gradiente.
  - O gradiente de uma função multivariada *f* é um **vetor** em que cada componente expressa a derivada parcial de *f* com respeito a uma das variáveis.
    - $\nabla L(f(x;\theta),y) = \begin{bmatrix} \frac{\partial}{\partial w_1} L(f(x;\ \theta),y) \\ \vdots \\ \frac{\partial}{\partial w_n} L(f(x;\ \theta),y) \\ \frac{\partial}{\partial b} L(f(x;\ \theta),y) \end{bmatrix}$
    - Relembrando que na fórmula acima, $\hat{y}$ foi representado como $f(x;\ \theta)$ para deixar clara sua dependência em relação a θ.
    - Conforme apresentado, a **equação final** para **atualizar** θ no passo $t+1$ é:
      - $\theta^{t+1} = \theta^{t} - \eta\left(\nabla L(f(x;\theta),y)\right)$

Gradiente Descendente – Intuição (6/8)

- **Mas como fica a derivada da função de perda?**
  - Em nosso exemplo, a função de perda é a SQE:

$$L(y, \hat{y}) = \sum_{k=1}^{m} \left(y^{(k)} - \hat{y}^{(k)}\right)^2$$

  - Por conveniência (*para combinar melhor com a derivação que será feita*), vamos dar uma pequena mexida e adotar metade da soma dos quadrados dos erros como função de perda:

$$L(y, \hat{y}) = \frac{1}{2} \sum_{k=1}^{m} \left(y^{(k)} - \hat{y}^{(k)}\right)^2$$

Gradiente Descendente – Intuição (7/8)

- **Mas como fica a derivada da função de perda?**

$$L(y,\hat{y}) = \frac{1}{2}\sum_{k=1}^{m}\left(y^{(k)} - \hat{y}^{(k)}\right)^2$$

- A derivada da função L acima para **uma observação x** em relação à variável j é:
  - $\frac{\partial L(\hat{y},y)}{\partial w_j} = \frac{1}{2} \times 2(\hat{y} - y)x_j = (\hat{y} - y)x_j$ ou $-(\boldsymbol{y} - \hat{\boldsymbol{y}})\boldsymbol{x_j}$
- Intuitivo!
  - O gradiente com respeito a um único peso w_j é a **diferença entre o y real e o ŷ estimado** para a observação, **multiplicado pelo valor de entrada correspondente** x_j.

Gradiente Descendente – Intuição (8/8)

- **Equação final**
  - Agora que sabemos calcular o gradiente da função de custo, a **equação final** para **atualizar** θ no passo t+1 é:
    - $\theta^{(t+1)} = \theta^{(t)} - \eta\left(\nabla L(f(x;\theta), y)\right),$
    - $\theta^{(t+1)} = \theta^{(t)} - \eta\left(-\left[y^{(t)} - \hat{y}^{(t)}\right] x^{(t)}\right),$
    - $\theta^{(t+1)} = \theta^{(t)} + \eta\left(e^{(t)} x^{(t)}\right)$

Treinamento do Perceptron – um exemplo (1/5)

- **Exemplo – Classificador de Sentimentos**
  - Vamos apresentar um exemplo de uma etapa de treinamento, que corresponde ao modelo recebendo a primeira observação da base de dados ao lado.
  - É uma BD de treino hipotética p/ classificação de sentimentos com duas variáveis:
    - x_1 → contagem de palavras positivas
    - x_2 → contagem de palavras negativas.
  - Com relação ao atributo classe y :
    - 0 → sentimento negativo
    - 1 → sentimento positivo

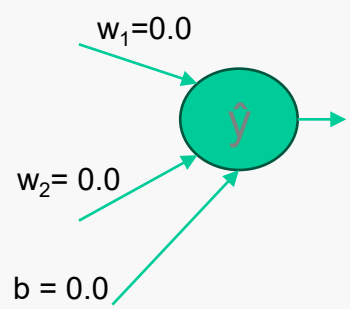
x_1 (contagem de palavras positivas)	x_2 (contagem de palavras negativas)	y (classe)
3	2	1
5	0	1
0	1	0
7	2	1
1	3	0
...	...	...
3	0	1

Treinamento do Perceptron – um exemplo (2/5)

- **Exemplo – Classificador de Sentimentos**
  - Vamos apresentar um exemplo de uma etapa de treinamento.
  - Será o modelo recebendo **a primeira** observação da base → [3, 2]
    - $x_1 = 3$ (contagem de palavras positivas)
    - $x_2 = 2$ (contagem de palavras negativas).
    - A classe real da observação é $y = 1$.

x_1 (contagem de palavras positivas)	x_2 (contagem de palavras negativas)	y (classe)
3	2	1
5	0	1
0	1	0
7	2	1
1	3	0
...	...	...
3	0	1

  - O exemplo vai mostrar o **primeiro passo do treinamento** do Perceptron. Considere a seguinte configuração:
    - $\theta^{(0)} = w_1 = w_2 = b = 0$ (pesos iniciais)
    - $\eta = 0.1$ (taxa de aprendizado)



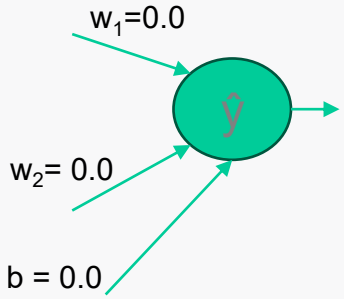
Treinamento do Perceptron – um exemplo (3/5)

- **Exemplo – Classificador de Sentimentos**
  - **Etapa de atualização p/ observação:** $\mathbf{x} = [3,2]$, $y = 1$.
  - **Sendo**: $\theta^{(0)} = w_1 = w_2 = b = 0.0$ $\qquad \eta = 0.1$

  - PASSO 1: considerando os parâmetros atuais $\theta^{(0)}$, e o vetor $\mathbf{x}$ atual, calculamos a saída $\hat{y}$ gerada pelo perceptron:
    - $z = w_1 x_1 + w_2 x_2 + b = 0.0 \times 3 + 0.0 \times 2 + 0.0 = 0.0$
    - $\hat{y} = \Gamma(z) = 0.0$

  - PASSO 2: calculamos o erro na saída:
    - $e = y - \hat{y} = 1 - 0 = 1.0$

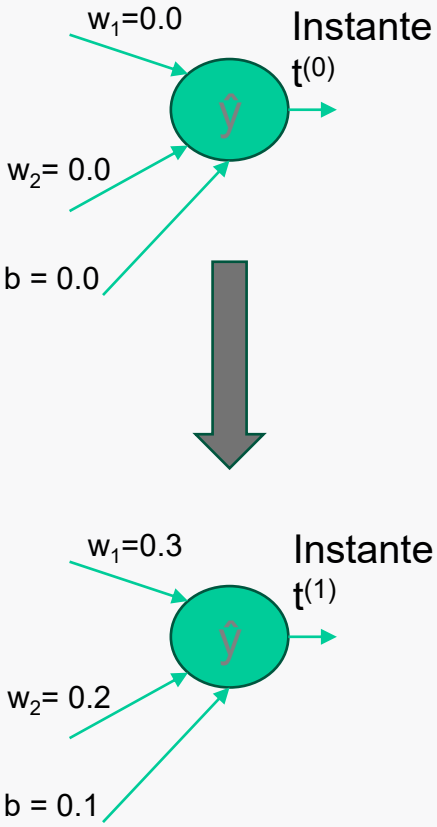
    - *Podemos ver que, para os parâmetros atuais, o Perceptron produziu uma saída diferente do valor real da classe.*



IBGE
Escola Nacional de Ciências Estatísticas ENCE

Treinamento do Perceptron – um exemplo (4/5)

- **Exemplo – Classificador de Sentimentos**
  - **Etapa de atualização p/ observação:** $x_1 = 3$, $x_2 = 2$, $y = 1$
  - **Sendo**: $\theta^{(0)} = w_1 = w_2 = b = 0$ $\eta = 0.1$
  - PASSO 3: cálculo dos novos valores de w_1 , w_2 e b
    - $w_1^{(1)} = w_1^{(0)} + \eta(e^{(0)} x_1^{(0)}) = 0 + 0.1 \times (1.0 \times 3) = 0.3$
    - $w_2^{(1)} = w_2^{(0)} + \eta(e^{(0)} x_2^{(0)}) = 0 + 0.1 \times (1.0 \times 2) = 0.2$
    - $b^{(1)} = b^{(0)} + \eta(e^{(0)}) = 0 + 0.1\ (1) = 0.1$
  - Novos parâmetros $\theta^{(1)}$:
    - $w_1 = 0.3$
    - $w_2 = 0.2$
    - $b = 0.1$
  - Ou seja: depois de 1 passo do GD, os parâmetros mudaram p/ $w_1 = 0.3$, $w_2 = 0.2$ e $b = 0.1$

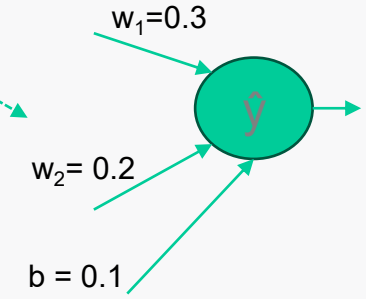


IBGE
Escola Nacional de Ciências Estatísticas ENCE

Treinamento do Perceptron – um exemplo (5/5)

- **Como o algoritmo vai continuar ?**
- O algoritmo prossegue pegando a <u>próxima observação</u>
- E realizando o mesmo processamento, mas agora com os <u>novos pesos</u>
- Quando processar a última observação da BD de treino, dizemos que acabou uma **época**.
- Então, ele vai para a 2ª época (processa toda a base de dados de novo). E fará isso:
  - por um número fixo de épocas (ex.: 1000 épocas)
  - ou até a SQE de uma época ficar muito baixa (abaixo de um limiar)
  - Ou até os pesos estarem mudando muito pouco (abaixo de um limiar)

x_1 (contagem de palavras positivas)	x_2 (contagem de palavras negativas)	y (classe)
3	2	1
5	0	1
0	1	0
7	2	1
1	3	0
...	...	...
3	0	1



IBGE
Escola Nacional de Ciências Estatísticas ENCE

A Nossa Tabela de Técnicas !!!

- **Tabela: propriedades das técnicas apresentadas em nosso curso**

Técnica	Principais Aplicações	Dimensão	Paradigma
***n*-Gramas de Caracteres**	*features* para modelos de identif. de plágio, detecção de idioma, atribuição de autor etc.; comparação de strings.	Morfológica	Simbólico
Bag of Words (BoW)	*features* para Naive Bayes e outros algoritmos	Morfológica	Simbólico*
Naïve Bayes Multinomial	análise de sentimentos, classificação de texto	Morfológica	Estatístico
Perceptron	classificação binária de texto	Fonética a Semântica**	Estatístico, Neural

* A BoW para um documento é gerada de acordo com uma regra pré-definida e não depende dos demais documentos da mesma base.
** Depende das *features* utilizadas. No exemplo dessa aula há alguma semântica envolvida devido ao uso de palavras negativas e positivas como *features*.

Referências

- JURAFSKY, D.; MARTIN, J. H. Logistic Regression. *In*: JURAFSKY, D.; MARTIN, J. H. **Speech and Language Processing**. 3. ed. *draft* [*s.l.*] : [*s.n.*], 2026. cap 4, p. 6-8. Disponível em: https://web.stanford.edu/~jurafsky/slp3/. Acesso em: 10 mar. 2026.

- DE OLIVEIRA, M. A.; MELO JR., F. A P. Perceptron. *In*: DE OLIVEIRA, M. A.; MELO JR., F. A P. **Backpropagation e Redes Neurais – Volume 1:** MLP, RPROP, LIMBP, DELTA-BAR-DELTA, MATLAB e PYTHON. Rio de Janeiro: Ciência Moderna, 2024. cap 3, p. 92-142.